SIMATS ENGINEERING

ASSESSMENT TOOL 3

Student Details	Information
Name	Mohammed Fareeth S A
Register Number	192511288
Course	Artificial Intelligence
Course Code	CSA17
Course Outcome	CO2: Search and game-solving techniques with heuristic functions

QUESTION 1 — BREADTH-FIRST SEARCH (BFS)

Graph: A is the root; A has children B and C; B has children D and E; C has children F and G. BFS uses a FIFO (first-in, first-out) queue and visits nodes level by level.

Iteration	Queue before removal	Visited node	Queue after expansion
1	[A]	A	[B, C]
2	[B, C]	B	[C, D, E]
3	[C, D, E]	C	[D, E, F, G]
4	[D, E, F, G]	D	[E, F, G]
5	[E, F, G]	E	[F, G]
6	[F, G]	F	[G]
7	[G]	G	[]

BFS traversal order

A → B → C → D → E → F → G

Explanation: Start with A. Remove the front node, visit it, and add its unvisited children to the rear of the queue. Continue until the queue is empty.

QUESTION 2 — DEPTH-FIRST SEARCH (DFS)

DFS explores as far as possible along one branch before backtracking. With children considered left to right, the traversal visits A, then B, then D. D has no children, so DFS backtracks to B and visits E. It then backtracks to A and explores C, followed by F and G.

Step	Node visited	Stack after processing (top at right)
1	A	[C, B]
2	B	[C, E, D]
3	D	[C, E]
4	E	[C]
5	C	[G, F]
6	F	[G]
7	G	[]

DFS traversal order: A → B → D → E → C → F → G

Implementation note: The displayed stack records the next nodes to explore, with the top at the right. To visit left children first using a stack, right children are pushed before left children.

QUESTION 3 — 8-PUZZLE USING BREADTH-FIRST SEARCH

Initial and goal states

Initial state	Goal state
1 3 6 5 0 2 4 7 8	1 2 3 4 5 6 7 8 0

The blank is represented by 0. A legal move swaps the blank with an adjacent tile (Up, Down, Left, or Right), provided the move stays inside the 3×3 board. BFS uses a FIFO queue and stores each discovered state only once.

BFS dry-run table (first five iterations)

Iteration	Current state (rows separated by /)	Queue before expansion	Queue after expansion
1	136/502/478	Front: 136/502/478; queued states: 1	Front: 106/532/478; queue size: 4
2	106/532/478	Front: 106/532/478; queued states: 4	Front: 136/572/408; queue size: 5
3	136/572/408	Front: 136/572/408; queued states: 5	Front: 136/052/478; queue size: 6
4	136/052/478	Front: 136/052/478; queued states: 6	Front: 136/520/478; queue size: 7
5	136/520/478	Front: 136/520/478; queued states: 7	Front: 016/532/478; queue size: 8

The worksheet provides only five table rows, but the goal is not reached within the first five expansions. BFS must continue until the goal state is removed from the queue. The table above records the first five actual expansions; the complete shortest path is shown below.

Complete shortest solution path

Minimum number of moves: 8

Blank-move sequence: Right → Up → Left → Down → Left → Down → Right → Right

Initial state:
1 3 6
5 0 2
4 7 8

State 1:
1 3 6
5 2 0
4 7 8

State 2:
1 3 0
5 2 6
4 7 8

State 3:
1 0 3
5 2 6
4 7 8

State 4:
1 2 3

5 0 6
4 7 8

State 5:
1 2 3
0 5 6
4 7 8

State 6:
1 2 3
4 5 6
0 7 8

State 7:
1 2 3
4 5 6
7 0 8

Goal state:
1 2 3
4 5 6
7 8 0

Which node is expanded in each iteration?

In the full BFS run, the node expanded in each iteration is the state at the front of the queue. Using the specified move order (Up, Down, Left, Right) and avoiding duplicate states, the goal state is dequeued at expansion 311. The first five expanded states are shown in the dry-run table.

How many states are generated in total?

With the stated move order and duplicate-state checking, 500 unique states have been discovered by the time the goal state is dequeued (including the initial and goal states). This count is implementation/order dependent: different legal-move ordering or counting repeated successor attempts can produce a different number. The shortest path itself remains 8 moves.

Why does BFS find the shortest solution in an unweighted 8-puzzle?

Every legal move has the same cost (one move). BFS explores all states at depth 0, then depth 1, then depth 2, and so on. Therefore, the first time it reaches the goal, no solution with fewer moves can exist; all shallower states have already been explored. This guarantee assumes the state space is searched systematically and repeated states are handled correctly.